

TEMA N° 2



TEMA 2 CONTENEDORES Y VIRTUALIZACIÓN LIGERA



EN ESTE TEMA SERÁS CAPAZ DE...

1. **Diferenciar con precisión** el modelo de virtualización tradicional basado en hipervisores de la virtualización ligera mediante contenedores, comprendiendo el ahorro de recursos de hardware en servidores reales.
2. **Dominar la arquitectura de Docker**, sus componentes principales (cliente, demonio, imágenes, contenedores y registros) y su funcionamiento interno bajo estándares de la industria.
3. **Instalar, configurar y operar entornos de contenedores** a través de la consola de comandos, desplegando aplicaciones web funcionales y gestionando su ciclo de vida como un Técnico preparado para las demandas del entorno laboral actual.



PRÁCTICA



¿Te has imaginado alguna vez cómo hacen las grandes empresas de desarrollo de software o los negocios en constante crecimiento para ejecutar decenas de aplicaciones diferentes en una sola computadora física sin que el sistema se vuelva extremadamente lento, inestable o colapse por falta de memoria RAM?

Imaginemos una situación real en nuestro **CEA Herman Ortiz Camargo**, ubicado aquí en la población de Pailón, en el Barrio Saó. El centro acaba de inaugurar un módulo educativo propio, moderno y equipado con computadoras destinadas al laboratorio de la carrera de Sistemas Informáticos. Como futuros Técnicos, se les ha encomendado la tarea de desarrollar y poner en marcha de forma simultánea múltiples proyectos informáticos para dinamizar la economía de los distintos barrios de nuestra región:

1. Un sistema de inventario y facturación para las tiendas comerciales del **Barrio Central Fátima**, que requiere una base de datos PostgreSQL y una aplicación construida en Python 3.10.
2. Un portal web interactivo para la gestión de campeonatos deportivos en las canchas del **Barrio 24 de Septiembre**, programado en Node.js 18 y con base de datos MySQL.
3. Una plataforma de venta de boletos y control de asistencia para los eventos culturales del coliseo municipal en el **Barrio 27 de Mayo**, desarrollada sobre un entorno PHP 8.2 clásico.
4. Un prototipo de sistema de monitoreo de seguridad vecinal para el **Barrio Residencial Norte**, que recopila alertas de cámaras locales.

El gran dilema técnico surge cuando intentamos instalar todas estas herramientas de software directamente sobre el sistema operativo de una sola computadora del laboratorio. Las versiones de las librerías entran en conflicto, los puertos de red chocan entre sí, y la instalación de bases de datos masivas satura el almacenamiento y la memoria RAM de los equipos.

Si decidiéramos solucionar esto utilizando Máquinas Virtuales tradicionales para cada barrio, cada una requeriría su propio sistema operativo completo (Windows o Linux), consumiendo un



mínimo de 2 a 4 GB de memoria RAM por proyecto. Con solo tres proyectos corriendo a la vez, las computadoras del laboratorio agotarían sus recursos de hardware, ralentizando el aprendizaje de todo el grupo de estudiantes.

¿Cómo podemos lograr que todos estos sistemas coexistan de forma totalmente aislada, segura y consumiendo el mínimo posible de memoria RAM y disco en las computadoras del CEA? La respuesta a este desafío local se encuentra en la adopción de los contenedores y la virtualización ligera.



TEORÍA



2.1. DIFERENCIAS ENTRE MÁQUINAS VIRTUALES TRADICIONALES Y CONTENEDORES

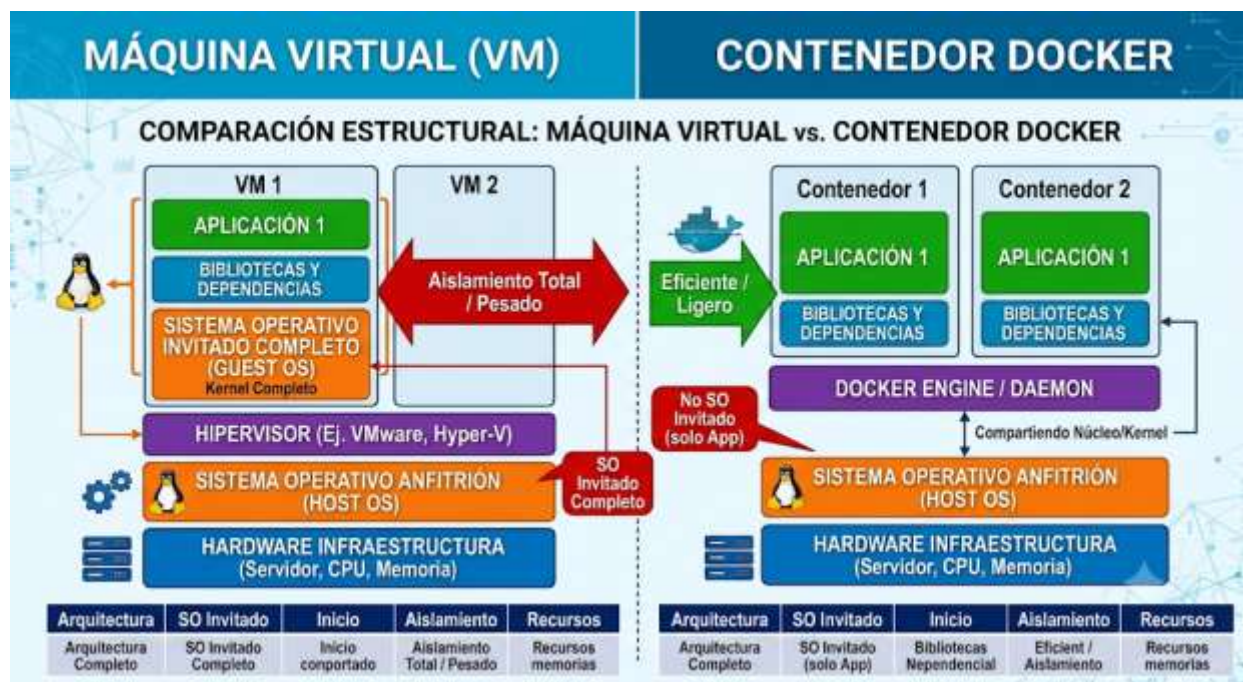
Para entender el valor de la virtualización ligera, es indispensable compararla con la virtualización tradicional que se ha venido utilizando en los servidores durante las últimas décadas. Ambas tecnologías buscan el mismo objetivo: aislar una aplicación y sus dependencias para que pueda ejecutarse en cualquier entorno sin interferir con otros programas. Sin embargo, la forma en que lo logran internamente es completamente distinta.

La **Virtualización Tradicional (Máquinas Virtuales)** se basa en una capa de software llamada **Hipervisor** (como VMware, VirtualBox o KVM). El hipervisor se instala sobre el hardware físico (o sobre un sistema operativo anfitrión) y tiene la tarea de simular hardware virtual independiente: CPUs virtuales, tarjetas de red virtuales, memoria RAM virtual y discos duros virtuales. Sobre este hardware simulado, el Técnico debe instalar un **Sistema Operativo Invitado (Guest OS)** completo para cada máquina virtual. Esto significa que si necesitas correr tres aplicaciones aisladas, tendrás tres sistemas operativos completos ejecutándose simultáneamente sobre la máquina física. Cada uno de estos sistemas operativos consume gigabytes de espacio en disco, requiere su propio gestor de memoria y tarda minutos en arrancar debido a su proceso de inicio completo (Kernel, servicios, controladores).

Por otro lado, la **Virtualización Ligera (Contenedores)** prescinde por completo del hipervisor y del sistema operativo invitado. En su lugar, se instala un **Motor de Contenedores** (como Docker)



directamente sobre el sistema operativo de la máquina física (normalmente Linux). Los contenedores son, en esencia, procesos aislados que se ejecutan directamente en el **Núcleo (Kernel)** del sistema operativo anfitrión. Gracias a tecnologías nativas del kernel de Linux, como los *Namespaces* (espacios de nombres que aíslan procesos, redes y usuarios) y los *Cgroups* (grupos de control que limitan el uso de CPU y memoria), un contenedor cree que tiene su propio sistema operativo dedicado, pero en realidad está compartiendo los recursos y el núcleo de la máquina real.



A continuación, se detalla una tabla comparativa técnica que resume estas diferencias críticas para el diseño de infraestructura:

Característica Técnica	Máquinas Virtuales Tradicionales	Contenedores (Virtualización Ligera)
Arquitectura Interna	Incluye un Sistema Operativo Invitado completo.	Comparte el Núcleo (Kernel) del S.O. Anfitrión.
Consumo de Memoria	Alto (Mínimo 1 GB a 4 GB por instancia).	Muy bajo (Pocos megabytes por instancia).
Espacio en Disco	Grande (Varios Gigabytes por archivo de disco virtual).	Pequeño (Megabytes, solo contiene la app y librerías).



Tiempo de Arranque	Lento (De 1 a 3 minutos para iniciar el S.O.).	Casi instantáneo (Fracciones de segundo).
Rendimiento de E/S	Menor, debido a la traducción del hipervisor.	Nativo, los procesos van directo al procesador real.
Portabilidad	Depende del formato del hipervisor (.ova, .vmdk).	Alta, se ejecuta igual en cualquier S.O. con Docker.

Como Técnicos en Sistemas Informáticos, entender esta diferencia nos permite ver por qué los contenedores son la opción ideal para optimizar el nuevo laboratorio del CEA Herman Ortiz Camargo, permitiendo exprimir al máximo el rendimiento del hardware disponible.

2.2. ARQUITECTURA, COMPONENTES Y FUNCIONAMIENTO DE DOCKER

Docker es actualmente la plataforma de software de código abierto líder para la creación, despliegue y gestión de contenedores. Para trabajar con Docker de manera profesional, es vital comprender su arquitectura basada en un modelo **Cliente-Servidor**. Esto significa que las herramientas con las que interactúa el Técnico no realizan el trabajo directamente, sino que envían peticiones a un servicio que se ejecuta en segundo plano.

Los componentes fundamentales de la arquitectura de Docker son los siguientes:

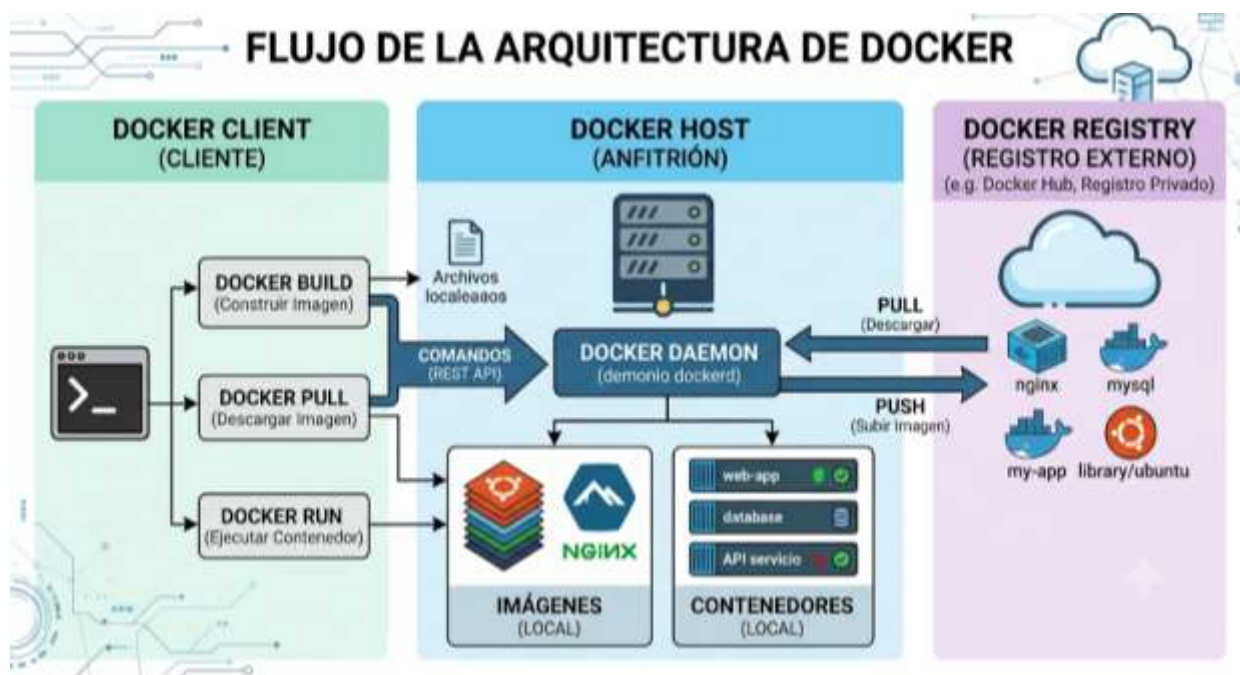
1. **Docker Daemon (dockerd):** Es el servidor o servicio que se ejecuta de forma continua en segundo plano en el sistema operativo anfitrión. Es el verdadero "cerebro" de Docker. Se encarga de escuchar las peticiones de la API de Docker y gestionar los objetos del sistema: crear los contenedores, descargar las imágenes del almacenamiento local, configurar las redes virtuales y conectar los volúmenes de datos persistentes.
2. **Docker Client (docker):** Es la interfaz de línea de comandos (CLI) con la que interactuamos los usuarios a través de la terminal. Cuando ejecutamos un comando como `docker run` o `docker ps`, el cliente traduce esta entrada en una petición de API REST y la envía al Docker Daemon para que la ejecute. El cliente puede comunicarse con un demonio local en la misma computadora o con un demonio Docker remoto a través de la red.
3. **Docker Registry (Registros):** Son repositorios remotos o locales encargados de almacenar y catalogar las imágenes de Docker. El registro público más grande y utilizado del mundo es **Docker Hub**, que viene configurado por defecto en la plataforma. Cuando



solicitamos una tecnología que no tenemos descargada, Docker busca automáticamente en este registro web.

4. Objetos de Docker:

1. **Imágenes:** Son plantillas de solo lectura que contienen las instrucciones estructuradas para crear un contenedor. Incluyen el código de la aplicación, el sistema de archivos base recortado, las variables de entorno y las librerías necesarias.
2. **Contenedores:** Son las instancias ejecutables y vivas de una imagen. Si la imagen es una clase en programación o un plano arquitectónico, el contenedor es el objeto instanciado o la casa ya construida. Se pueden iniciar, detener, mover o eliminar mediante comandos rápidos.
3. **Redes (Networks):** Canales virtuales creados por Docker para permitir que los contenedores se comuniquen entre sí o con el mundo exterior de forma segura y aislada.
4. **Volúmenes (Volumes):** Mecanismos de almacenamiento diseñados para persistir los datos generados por los contenedores. Como los contenedores son efímeros (si se borran, sus datos internos desaparecen), los volúmenes enlazan carpetas del contenedor con carpetas reales de la computadora física anfitriona.





El funcionamiento sigue una lógica sencilla: el cliente solicita una acción, el demonio comprueba si dispone de los recursos locales (como las imágenes); si no los tiene, los descarga del registro, y finalmente ensambla y arranca el contenedor de manera totalmente aislada del resto del sistema operativo.

2.3. INSTALACIÓN DE DOCKER Y COMANDOS ESENCIALES BÁSICOS

Para poner en práctica la virtualización ligera en el laboratorio del CEA Herman Ortiz Camargo, el Técnico debe preparar el sistema operativo. Aunque Docker se puede instalar en sistemas Windows y macOS a través de Docker Desktop, en entornos profesionales y servidores de producción se implementa de forma nativa sobre distribuciones Linux como Ubuntu Server o Debian, maximizando la eficiencia energética y el rendimiento.

A continuación, se detalla el bloque secuencial de comandos para instalar el motor de Docker (Docker Engine) en un sistema basado en Ubuntu/Debian. Cada instrucción está comentada detalladamente para comprender su propósito dentro del sistema operativo:

Bash

```
# Actualiza el índice de paquetes Locales para asegurar que descargaremos las versiones más recientes
sudo apt-get update
# Instala paquetes prerequisites para permitir que ' apt
' descargue repositorios de forma segura sobre HTTPS
sudo apt-get install -y apt-transport-https ca-certificates curl software-properties-common
# Descarga e incorpora la clave criptográfica GPG oficial de Docker para validar la autenticidad de las descargas
curl -fsSL https://download.docker.com/linux/ubuntu/gpg | sudo gpg --dearmor -o /usr/share/keyrings/docker-archive-keyring.gpg
# Agrega de forma permanente el repositorio oficial de Docker estable a las fuentes de software de nuestro sistema
echo "deb [arch=$(dpkg --print-architecture) signed-by=/usr/share/keyrings/docker-archive-keyring.gpg] https://download.docker.com/linux/ubuntu (lsb_release -cs) stable" | sudo tee /etc/lists.d/docker.list > /dev/null
# Vuelve a actualizar el índice de paquetes para incluir el nuevo repositorio oficial de Docker recién añadido
sudo apt-get update
# Instala el paquete del motor de Docker, la interfaz de comandos (CLI) y el runtime
sudo apt-get install -y docker-ce docker-ce-cli containerd.io
# Habilita el servicio de Docker para que se inicie automáticamente cada vez que se encienda la computadora
sudo systemctl enable docker
```



```
# Arranca de forma inmediata el demonio de Docker en segundo plano  
sudo systemctl start docker
```

Una vez completada con éxito la instalación, el Técnico interactúa con el demonio mediante los comandos esenciales básicos. Estos comandos conforman el núcleo de la administración diaria de infraestructuras basadas en contenedores:

Bash

```
# Verifica el estado del servicio y comprueba la versión exacta instalada de cliente y servidor  
docker version  
# Muestra información detallada sobre todo el sistema de Docker (número de contenedores, imágenes y recursos del host)  
docker info  
# Lista todos los contenedores que se encuentran activos y ejecutándose en este preciso instante en la máquina  
docker ps  
# Lista absolutamente todos los contenedores en el sistema, sin importar si están activos, pausados o detenidos  
docker ps -a  
# Detiene de forma controlada la ejecución de un contenedor en marcha utilizando su ID o su nombre asignado  
docker stop nombre_del_contenedor  
# Inicia un contenedor que previamente había sido detenido, manteniendo su estado y configuraciones internas  
docker start nombre_del_contenedor  
# Elimina definitivamente un contenedor del almacenamiento local (el contenedor debe estar detenido para poder borrarlo)  
docker rm nombre_del_contenedor
```

2.4. DESCARGA Y GESTIÓN DE IMÁGENES DESDE REPOSITORIOS (DOCKER HUB)

Las imágenes de Docker son las piezas fundamentales de software empaquetado que descargamos para dar vida a los contenedores. Como se mencionó anteriormente, cuando requerimos una herramienta de software, Docker Hub actúa como la tienda o biblioteca centralizada mundial. Las imágenes en Docker Hub están organizadas bajo una nomenclatura estándar: nombre_de_la_imagen:etiqueta (por ejemplo, ubuntu:22.04 o postgres:15-alpine). La etiqueta (*tag*) suele indicar la versión específica del software o del sistema operativo base; si no se especifica ninguna etiqueta, Docker asumirá automáticamente el modificador :latest, descargando la versión más reciente disponible.

Internamente, las imágenes están compuestas por una serie de **capas de solo lectura superpuestas**. Cada capa representa una instrucción o cambio en el sistema de archivos (como la instalación de una librería o la adición de un archivo de configuración). Esto permite una



característica revolucionaria: la **reutilización de capas**. Si descargas tres imágenes distintas que utilizan una base común de Ubuntu, esa base se descarga una sola vez en el disco duro del servidor del CEA, ahorrando un valioso espacio de almacenamiento y acelerando drásticamente las descargas futuras.

Para gestionar estas imágenes de forma profesional en nuestra consola, empleamos los siguientes comandos fundamentales:

Bash

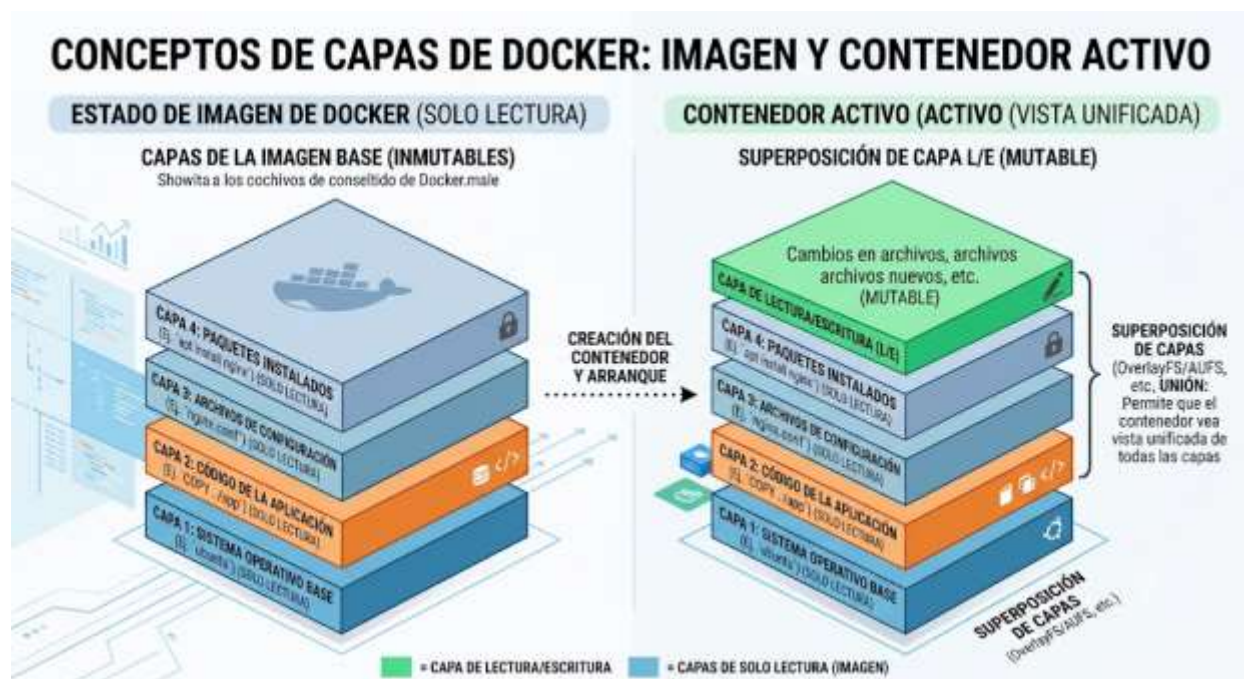
```
# Busca en el registro público de Docker Hub imágenes relacionadas con una
palabra clave, por ejemplo, bases de datos MySQL
docker search mysql

# Descarga de forma explícita una imagen desde Docker Hub al almacenamiento
local de la computadora sin llegar a ejecutarla todavía
docker pull nginx:alpine

# Muestra un listado completo de todas las imágenes de software que han sido
descargadas y guardadas localmente en el host
docker images

# Muestra información técnica detallada en formato JSON de una imagen específica
(capas, variables de entorno, puertos expuestos)
docker inspect nginx:alpine

# Borra de forma definitiva una imagen del almacenamiento local para liberar
espacio en el disco duro del servidor
docker rmi nginx:alpine
```





Cuando un contenedor se ejecuta a partir de una imagen, Docker añade una delgada capa superior de **lectura y escritura** específica para esa instancia de contenedor. Cualquier modificación, archivo creado o base de datos modificada se guarda únicamente en esa capa superior efímera, dejando la imagen base completamente intacta y lista para generar más contenedores idénticos.

2.5. DESPLIEGUE DE UN CONTENEDOR BÁSICO DE PRUEBA

Ha llegado el momento de consolidar la teoría uniendo todos los conceptos aprendidos en una acción práctica real: el despliegue de nuestro primer contenedor de prueba. Para este laboratorio inicial, utilizaremos una imagen oficial de **Nginx**, uno de los servidores web más rápidos, estables y ligeros del mercado mundial, ampliamente utilizado para servir páginas HTML estáticas y actuar como proxy inverso.

El comando definitivo para descargar, configurar y poner en marcha el servidor web de manera simultánea en una sola línea de comandos es el siguiente:

Bash

```
# Despliega un contenedor web Nginx totalmente aislado en segundo plano mapeando  
puertos de red externos docker run --name servidor-web-pailon -d -p 8080:80  
nginx:alpine
```

Analicemos minuciosamente cada parámetro de este comando, lo cual representa un conocimiento técnico indispensable para cualquier examen o despliegue en producción:

1. `docker run`: Es el comando compuesto del cliente Docker que ordena al demonio realizar dos acciones consecutivas: si la imagen especificada no existe localmente en el host, realiza un `docker pull` automático para traerla de Docker Hub, y acto seguido crea e inicia el contenedor ejecutable.
2. `--name servidor-web-pailon`: Este modificador opcional pero altamente recomendado asigna un nombre amigable e identificable a la instancia del contenedor en el sistema. Si omitimos este parámetro, Docker inventará un nombre aleatorio compuesto por el nombre de un científico famoso y un adjetivo (por ejemplo, `boring_einstein`), lo cual dificulta la administración posterior.



3. -d: Significa ejecutar en modo **Detached** (desacoplado o en segundo plano). Al incluir este flag, el contenedor se inicia de forma silenciosa en el trasfondo del sistema operativo, liberando inmediatamente nuestra terminal de comandos para que podamos seguir escribiendo otras instrucciones sin interrumpir el proceso del servidor.
4. -p 8080:80: Configura el **Mapeo o Redirección de Puertos (Port Forwarding)**. El formato estricto es PuertoHost:PuertoContenedor. El contenedor ejecuta internamente Nginx escuchando en su propio puerto nativo 80 (dentro de su red aislada). Al indicarle 8080:80, estamos abriendo el puerto 8080 en la tarjeta de red real de nuestra computadora física del CEA, y ordenando que todo el tráfico de red que llegue allí sea redirigido de forma transparente al puerto 80 interno del contenedor.
5. nginx:alpine: Especifica el nombre de la imagen y su etiqueta. La palabra clave alpine hace referencia a una distribución de Linux extremadamente minimalista (Alpine Linux), cuyo tamaño total en disco ronda apenas los 5 MB, garantizando un despliegue ultrarrápido y un consumo mínimo de recursos de hardware.

Para verificar que nuestro servidor web local está funcionando correctamente desde la misma terminal o desde cualquier otra computadora conectada a la red local del aula en Pailón, ejecutamos una petición HTTP básica:

Bash

```
# Realiza una petición de prueba al puerto mapeado del host para recibir la estructura HTML del servidor webcurl http://localhost:8080
```

Si todo ha sido configurado correctamente por el Técnico, la consola nos devolverá el código HTML con el mensaje de bienvenida de Nginx, confirmando que la virtualización ligera se ha ejecutado con éxito total en menos de un segundo.

2.6. ORQUESTACIÓN BÁSICA DE CONTENEDORES Y MICROSERVICIOS EN EL ENTORNO LOCAL

En el panorama tecnológico moderno, las aplicaciones reales raramente consisten en un único contenedor aislado. Siguiendo la arquitectura de **Microservicios**, los sistemas de software se



dividen en pequeños componentes especializados independientes que colaboran entre sí. Por ejemplo, el sistema de inventario comercial planeado para el *Barrio Central Fátima* requiere de forma obligatoria un contenedor para la interfaz web (Frontend), otro para la lógica de negocio (Backend) y un tercero exclusivo para almacenar los registros de datos (Base de Datos).

Administrar estos sistemas multicontenedor ejecutando comandos individuales docker run se vuelve complejo y propenso a errores humanos, ya que el Técnico debe enlazar manualmente las redes y controlar el orden de encendido de cada pieza. Para resolver este reto, la virtualización ligera introduce el concepto de **Orquestación Básica Local** utilizando una herramienta oficial llamada **Docker Compose**.

Docker Compose permite definir de forma declarativa toda una arquitectura de microservicios dentro de un único archivo de texto plano estructurado con formato YAML denominado docker-compose.yml. A través de este archivo, configuramos los contenedores, sus dependencias mutuas, variables de entorno secretas, redes internas protegidas y volúmenes persistentes.

Veamos un ejemplo avanzado y real de cómo estructurar un archivo de configuración para un microservicio web conectado a una base de datos PostgreSQL, diseñado para el control de los comercios en el Barrio Central Fátima de nuestra localidad:

YAML

```
# Especifica la versión de la sintaxis del archivo de Docker Compose a
utilizarversion: '3.8'
# Define el listado de servicios (contenedores independientes) que componen la
arquitectura de la aplicaciónservices: # Servicio 1: El motor de base de datos
relacional base-datos-fatima: image: postgres:15-alpine # Descarga una
imagen optimizada y muy ligera de PostgreSQL 15 container_name:
db_comercio_fatima # Asigna un nombre fijo al contenedor en el sistema
operativo environment: POSTGRES_USER: administrador_cea # Define el
nombre del usuario administrador del motor de datos POSTGRES_PASSWORD:
ClaveSeguraPailon2026 # Establece la contraseña de acceso al motor
POSTGRES_DB: bd_inventario_local # Crea automáticamente una base de datos con
este nombre al arrancar volumes: -
datos_postgres:/var/lib/postgresql/data # Mapea un volumen persistente para no
perder los datos al apagar el contenedor networks: - red-comercial-
interna # Conecta este contenedor a una red interna aislada y segura #
Servicio 2: La aplicación web de gestión que interactúa con el usuario sistema-
web-inventario: image: python:3.10-slim # Utiliza una imagen base oficial de
Python recortada para ahorrar espacio container_name: web_inventario_fatima
```



```
# Asigna el nombre al contenedor de la interfaz web    ports:      - "8000:8000"
# Mapea el puerto 8000 de la computadora del aula con el puerto 8000 interno de
La app Python    environment:    DATABASE_HOST: base-datos-fatima # Docker
resuelve automáticamente el nombre del servicio como si fuera una IP en la red
DATABASE_USER: administrador_cea    DATABASE_PASSWORD: ClaveSeguraPailon2026
DATABASE_NAME: bd_inventario_local    depends_on:      - base-datos-fatima #
Ordena a Docker Compose levantar obligatoriamente la base de datos antes de
iniciar la app web    networks:      - red-comercial-interna # Une la
aplicación a la misma red para comunicarse directamente con PostgreSQL
# Declaración explícita de los recursos globales independientes compartidos por
los servicios arriba descritosvolumes: datos_postgres: # Crea un volumen
administrado en el disco duro del host para almacenar los datos comerciales
realesnetworks: red-comercial-interna: # Crea una red virtual aislada donde
solo estos dos contenedores pueden verse entre sí
```

Para operar toda esta infraestructura simultáneamente, el Técnico en Sistemas Informáticos ya no necesita escribir comandos interminables. Se posiciona con la terminal de comandos en la carpeta donde guardó el archivo y ejecuta la directiva de orquestación local:

Bash

```
# Descarga las imágenes necesarias, crea las redes, volúmenes e inicia todos los
servicios del archivo en segundo plano docker compose up -d
# Muestra el estado operativo de los microservicios definidos exclusivamente
dentro del archivo YAML de esta carpeta docker compose ps
# Detiene de forma ordenada y destruye los contenedores, redes virtuales creadas
por el comando up, liberando el sistema por completo docker compose down
```

Esta abstracción reduce drásticamente el tiempo necesario para desplegar laboratorios complejos dentro de nuestro entorno educativo, unificando la administración en comandos simples y estandarizados.

2.7. OPTIMIZACIÓN Y SEGURIDAD DE CONTENEDORES ASISTIDA POR INTELIGENCIA ARTIFICIAL

A medida que el ecosistema de la virtualización ligera madura, la integración de herramientas de **Inteligencia Artificial (IA)** se ha vuelto una práctica habitual y de alto valor para los administradores de sistemas y Técnicos de soporte. En la actualidad, la IA asiste activamente en dos áreas críticas: la optimización del tamaño de las imágenes y la detección automatizada de brechas de seguridad dentro de las capas de software.



Durante el proceso de creación de imágenes personalizadas mediante archivos de configuración conocidos como Dockerfile, es común que los desarrolladores principiantes dejen residuos innecesarios de la instalación, como cachés de paquetes, archivos temporales de compilación o herramientas de depuración pesadas. Esto da como resultado imágenes infladas de más de 1 GB. Los modelos de IA aplicados al análisis de código estático actúan examinando el Dockerfile línea por línea, sugiriendo técnicas avanzadas como las **Construcciones Multietapa (Multi-stage Builds)**. Esta técnica permite utilizar una imagen robusta llena de herramientas pesadas únicamente para compilar la aplicación, y luego copiar exclusivamente el archivo ejecutable binario resultante dentro de una imagen base final ultraligera de solo 5 MB, reduciendo la superficie de almacenamiento y acelerando los despliegues de red en zonas con conectividad limitada.

En el ámbito de la seguridad informática, las herramientas basadas en IA analizan en tiempo real los metadatos y las capas de las imágenes de Docker contrastándolas con bases de datos globales de **Vulnerabilidades y Exposiciones Comunes (CVE)**. Tecnologías de escaneo automatizado asistido como *Trivy* o los motores integrados de Docker Scout permiten escanear una imagen antes de enviarla a un servidor real, identificando librerías desactualizadas o contraseñas quemadas accidentalmente en el código.

Por ejemplo, un Técnico puede ejecutar un escaneo automatizado en la consola de comandos del laboratorio para evaluar la seguridad de una aplicación de forma inmediata:

Bash

```
# Utiliza la herramienta de escaneo nativa de Docker para analizar vulnerabilidades en una imagen de base de datosdocker scout cves postgres:15-alpine
```

La IA evalúa los resultados, los categoriza según su nivel de criticidad (Baja, Media, Alta, Crítica) y ofrece recomendaciones de remediación inmediatas, sugiriendo la línea exacta de código que se debe actualizar. Esta capa de asistencia dota al futuro profesional de herramientas de nivel empresarial para garantizar entornos informáticos robustos, estables y protegidos contra ciberataques.



2.8. EL FUTURO DE LA VIRTUALIZACIÓN LIGERA: EDGE COMPUTING Y GREEN IT

El horizonte tecnológico de los contenedores se está expandiendo con fuerza más allá de los centros de datos masivos en la nube, apuntando a dos tendencias globales determinantes para el desarrollo tecnológico regional: el **Edge Computing (Computación en el Borde)** y el **Green IT (Tecnologías Verdes Sustentables)**.

El *Edge Computing* propone procesar los datos lo más cerca posible del lugar físico donde se generan, en lugar de enviar toneladas de información bruta a servidores remotos ubicados al otro lado del planeta. En localidades como Pailón, donde el ancho de banda de internet puede sufrir intermitencias climáticas o limitaciones de infraestructura, el Edge Computing se vuelve vital. Los contenedores, debido a su naturaleza ligera y tamaño reducido, se adaptan perfectamente para ejecutarse en dispositivos de hardware de bajo consumo y potencia limitada instalados localmente, como microcomputadoras Raspberry Pi o pasarelas de automatización basadas en arquitecturas de procesadores ARM. Un Técnico puede desplegar contenedores con algoritmos de analítica o servidores de bases de datos locales directamente en una antena de telecomunicaciones o en una pequeña oficina de cobranzas de un barrio alejado, garantizando que el sistema siga operando perfectamente de forma autónoma aunque la conexión troncal a internet se caiga por completo.

Directamente entrelazado con esto se encuentra el paradigma del *Green IT* o informática ecológica. El consumo de energía eléctrica de los servidores a nivel mundial representa una de las huellas de carbono de mayor crecimiento en el planeta. La virtualización tradicional desperdicia enormes cantidades de electricidad debido al sobreprocesamiento constante que requiere mantener encendidos docenas de sistemas operativos invitados redundantes que duplican funciones del kernel.

Al migrar hacia la virtualización ligera con contenedores, eliminamos por completo esa sobrecarga innecesaria de software. Un solo procesador real puede alojar hasta cuatro veces más aplicaciones en contenedores que en máquinas virtuales tradicionales utilizando exactamente la misma cantidad de vatios de energía eléctrica. Para nuestro CEA Herman Ortiz Camargo, esto se traduce de forma tangible en una reducción inmediata del consumo de electricidad en el nuevo módulo educativo, extendiendo la vida útil de los componentes de hardware de las



computadoras del aula al disminuir la generación de calor y el estrés térmico, contribuyendo de esta forma al desarrollo de una tecnología comunitaria sustentable y respetuosa con el medio ambiente local.

CIERRE TEÓRICO OBLIGATORIO

❑ Mitos vs. Realidades

Mito Tecnológico Común	Realidad Técnica Demostrable
Mito 1: Los contenedores Docker son idénticos a las Máquinas Virtuales pequeñas y se administran exactamente igual en producción.	Realidad: Los contenedores no simulan hardware ni ejecutan un sistema operativo invitado; son procesos aislados nativos que comparten el mismo núcleo del sistema operativo anfitrión.
Mito 2: Ejecutar aplicaciones dentro de contenedores disminuye drásticamente el rendimiento de los programas debido a la capa de empaquetado.	Realidad: La penalización de rendimiento en contenedores es cercana al 0%, ya que los procesos se ejecutan a velocidad nativa directamente sobre la CPU real sin traducción intermedia del hipervisor.

❑ Glosario de Términos

1. **Docker Daemon (dockerd):** Servicio en segundo plano que se ejecuta de forma persistente en el sistema operativo anfitrión encargado de construir, gestionar y controlar la totalidad de los objetos de Docker.
2. **Imagen de Docker:** Plantilla inmutable estructurada por capas de solo lectura que contiene el código fuente, librerías, dependencias y configuraciones necesarias para instanciar un contenedor ejecutable.
3. **Contenedor:** Instancia viva, aislada y ejecutable en memoria RAM de una imagen de Docker que funciona de forma independiente sin interferir con otros procesos del sistema.
4. **Maapeo de Puertos:** Técnica de configuración de redes virtuales que enlaza un puerto de la tarjeta de red de la computadora física con un puerto interno específico del contenedor, permitiendo el tráfico externo.



5. **Namespaces (Espacios de Nombres):** Característica nativa fundamental del núcleo de Linux que proporciona aislamiento visual de recursos (redes, procesos, ID de usuarios) a nivel de sistema operativo para cada contenedor.

□ **Ponte a Prueba**

1. **¿Cuál es el componente de la arquitectura de Docker encargado de escuchar las peticiones de la API de comandos y realizar el trabajo pesado de gestionar contenedores e imágenes en el sistema operativo?**

1. A) Docker Client
2. B) Docker Hub
3. C) Docker Daemon
4. *Respuesta Correcta: C*

2. **Si necesitas que los datos generados por una base de datos MySQL dentro de un contenedor no se borren de forma definitiva cuando el contenedor se detenga o elimine, ¿qué recurso de Docker debes configurar obligatoriamente?**

1. A) Una Red Virtual (Network)
2. B) Un Volumen Persistente (Volume)
3. C) Un nuevo Docker Client
4. *Respuesta Correcta: B*

3. **¿Qué parámetro se le debe pasar obligatoriamente al comando docker run en la terminal para indicar que el contenedor debe ejecutarse de forma silenciosa en segundo plano (modo Desacoplado o Detached)?**

1. A) -p
2. B) --name
3. C) -d



4. Respuesta Correcta: C



VALORACIÓN



La adopción de la virtualización ligera mediante contenedores nos invita como Técnicos en Sistemas Informáticos a una reflexión ética, social y medioambiental profunda sobre cómo ejercemos nuestra profesión en el contexto local y global.

Desde la perspectiva del **Impacto Social y de Democratización Tecnológica**, los contenedores representan una herramienta de inclusión revolucionaria. Tradicionalmente, para enseñar infraestructuras de red avanzadas o desplegar múltiples sistemas corporativos se requería una inversión económica prohibitiva en hardware de servidores de gran potencia, algo fuera del alcance de muchas instituciones educativas públicas del área rural o de pequeñas poblaciones en desarrollo. Al reducir el consumo de memoria RAM y almacenamiento a una fracción mínima, la virtualización ligera permite convertir computadoras de escritorio estándar en minicentros de datos capaces de ejecutar servicios comunitarios complejos. Esto rompe la brecha digital, permitiendo que desde el aula de nuestro CEA en Pailón podamos construir y probar soluciones tecnológicas con el mismo estándar de calidad con el que operan las grandes corporaciones en Silicon Valley o Europa, potenciando la soberanía tecnológica local.

En el **Ámbito Medioambiental**, el uso eficiente de los recursos informáticos se enlaza de forma directa con la mitigación de la problemática global del **e-waste (basura electrónica)**. El ciclo de obsolescencia impuesto por el mercado de software obliga constantemente a desechar computadoras funcionales simplemente porque los nuevos sistemas operativos pesados o las actualizaciones de hipervisores ya no pueden arrancar en procesadores de generaciones anteriores. Los contenedores, al ser tan optimizados y prescindir de sistemas operativos invitados redundantes, extienden de manera significativa el ciclo de vida útil del hardware existente. Una computadora que se consideraría obsoleta para ejecutar tres sistemas operativos virtuales modernos puede correr con total fluidez una docena de contenedores ligeros de Docker durante muchos años más. Esto reduce directamente la demanda de sustitución de hardware,



disminuyendo el volumen de componentes plásticos y metales pesados altamente contaminantes que terminan acumulándose en basureros a cielo abierto en nuestras regiones.

Finalmente, en el eje de la **Seguridad de la Información y la Privacidad**, la contención ligera nos obliga a ejercer una responsabilidad ética rigurosa en el manejo de datos ciudadanos. Compartir el mismo núcleo del sistema operativo entre múltiples contenedores implica que un error crítico de configuración o una vulnerabilidad grave no detectada en una capa del software podría ser aprovechada por usuarios malintencionados para realizar un "escape de contenedor" y acceder al sistema operativo anfitrión físico, comprometiendo los datos de todos los proyectos alojados. Como Técnicos, el despliegue de software no debe enfocarse nunca únicamente en lograr que un sistema funcione de forma rápida, sino en aplicar auditorías constantes, parches de seguridad oportunos y configuraciones bajo el principio ético del menor privilegio posible. La tecnología de contenedores nos dota de un poder de despliegue sin precedentes; el uso ético, consciente y seguro de ese poder es el sello de excelencia que debe caracterizar a los profesionales formados en nuestro centro educativo.



PRODUCCIÓN



LABORATORIO PRÁCTICO PASO A PASO: DESPLIEGUE DEL PORTAL DE LA "FERIA PRODUCTIVA DE PAILÓN" MEDIANTE CONTENEDORES Y VOLÚMENES DE DOCKER

Objetivo General del Laboratorio

El estudiante configurará, desplegará y validará un entorno web ligero y personalizado utilizando el servidor web Nginx empaquetado en un contenedor Docker. Aprenderá a utilizar volúmenes administrados para inyectar un sitio web dinámico propio, simulando un entorno de producción real para promocionar a los artesanos y productores locales de los barrios de nuestra población.

Requisitos Previos

1. Computadora con sistema operativo basado en Linux o Windows con Docker Engine instalado y en ejecución.



2. Terminal de comandos (Consola o PowerShell) abierta con privilegios de administrador.
3. Conexión a internet estable para la descarga inicial de la imagen desde Docker Hub.

PASO 1: Creación del espacio de trabajo local en la computadora del Host

Para organizar nuestro proyecto sin alterar otras carpetas del sistema operativo físico, crearemos un directorio dedicado exclusivo en el disco duro de nuestra computadora para almacenar la estructura web de la feria productiva:

Bash

```
# Crea una carpeta específica para el proyecto de La Feria de Pailón en el directorio del usuario mkdir -p HOME/Laboratorio-feria-pailon  
# Cambia el directorio de trabajo de la terminal para posicionarse dentro de la carpeta recién creada cd HOME/Laboratorio-feria-pailon
```

PASO 2: Creación de la página web estática personalizada para la comunidad

Utilizando el editor de texto integrado de la consola (como nano), generaremos un archivo HTML básico estructurado que actuará como la página de inicio de nuestro portal local. Ejecuta el comando para abrir el editor:

Bash

```
# Abre el editor nano para crear y editar el archivo index.html de nuestro portal web nano index.html
```

Ahora, copia y pega el siguiente código estructurado HTML directamente dentro de la interfaz del editor nano:

HTML

```
<!DOCTYPE html>  
<html lang="es">  
<head> <meta charset="UTF-8"> <meta name="viewport" content="width=device-width, initial-scale=1.0"> <title>Feria Productiva - CEA Herman Ortiz Camargo</title> <style> body { font-family: '
```



```

Segoe UI', Tahoma, Geneva, Verdana, sans-serif;
background-color: #f4f7f6;
margin: 0;
padding: 20px;
color: #333;
}
header {
background-color: #0b5a3e;
color: white;
padding: 25px;
text-align: center;
border-radius: 8px;
}
.contenedor {
max-width: 900px;
margin: 20px auto;
background: white;
padding: 30px;
border-radius: 8px;
box-shadow: 0 4px 6px rgba(0,0,0,0.1);
}
h2 {
color: #0b5a3e;
border-bottom: 2px solid #0b5a3e;
padding-bottom: 5px;
}
.barrio-card {
background: #eef7f2;
border-left: 5px solid #0b5a3e;
padding: 15px;
margin: 15px 0;
border-radius: 0 4px 4px 0;
}
footer {
text-align: center;
margin-top: 30px;
font-size: 0.9em;
color: #777;
}
</style>
</head>
<body>    <header>        <h1>Portal Digital de la Feria Productiva de
Pailón</h1>        <p>Desplegado exitosamente sobre Tecnologías de
Virtualización Ligera (Docker)</p>    </header>    <div class="
contenedor">        <h2>Sistemas Informáticos del CEA Herman Ortiz Camargo</h2>
<p>Este sitio web está siendo servido de forma nativa desde un contenedor
aislado de Nginx, compartiendo recursos del sistema de forma ecológica y
eficiente.</p>        <h2>Proyectos de Reactivación Comercial en los
Barrios:</h2>        <div class="

```



```
barrio-card">          <strong>Barrio Central Fátima:</strong> Sistema de
gestión de inventarios y facturación digital para los comercios locales frente a
la iglesia principal.          </div>          <div class="
barrio-card">          <strong>Barrio 24 de Septiembre:</strong> Plataforma de
reservas y control de estadísticas deportivas para los campos recreativos
cercanos a la estación de tren.          </div>          <div class="
barrio-card">          <strong>Barrio Saó:</strong> Módulo de control de
asistencia de estudiantes y gestión académica central del nuevo edificio propio
del CEA.          </div>          </div>          <footer>          <p>© 2026 - Diseñado y
Administrado por el Técnico en Sistemas Informáticos del CEA</p>          </footer>
</body>
</html>
```

Para guardar los cambios en el editor nano: Presiona la combinación de teclas Control + O, confirma el nombre presionando Intro o Enter, y finalmente sal del editor presionando Control + X.

PASO 3: Despliegue del contenedor web conectando los archivos locales mediante volúmenes

Ahora ejecutaremos el comando para levantar el contenedor. Utilizaremos el parámetro -v (Volume) para enlazar nuestra carpeta local recién creada con el directorio interno de Nginx encargado de servir los archivos web públicos hacia internet (/usr/share/nginx/html):

Bash

```
# Despliega el contenedor mapeando el puerto 80 y enlazando el archivo
index.html local mediante volúmenes del sistemadocker run --name web-feria-
pailon -d -p 80:80 -v HOME/Laboratorio-feria-pailon:/usr/share/nginx/html
nginx:alpine
```

Explicación técnica del comando ejecutado:

1. -v HOME/laboratorio-feria-pailon:/usr/share/nginx/html: Monta de forma interactiva la carpeta de nuestro host físico dentro del contenedor. Cuando Nginx intente buscar su página por defecto, leerá en tiempo real nuestro archivo index.html modificado. Esto significa que cualquier cambio que hagamos en el archivo local se verá reflejado inmediatamente en la web sin necesidad de reiniciar o reconstruir el contenedor.



PASO 4: Verificación y Auditoría del Estado del Contenedor

Como Técnicos encargados del soporte de la infraestructura, debemos validar el correcto funcionamiento y la salud del servicio desplegado mediante comandos de diagnóstico:

Bash

```
# Lista los contenedores en ejecución para constatar que '
web-feria-pailon
' se encuentra en estado UPdocker ps
# Inspecciona los logs o registros de acceso del contenedor en tiempo real para
visualizar las peticiones de los usuariosdocker logs web-feria-pailon
```

PASO 5: Prueba de Acceso Final al Sistema

Abre el navegador web preferido en tu computadora de escritorio (Chrome, Firefox, Edge) y escribe con precisión la siguiente dirección en la barra de navegación superior:

Plaintext

```
http:
//localhost
```

¡Felicidades! Verás aparecer de forma inmediata la interfaz del portal web de la Feria Productiva de Pailón adaptada al CEA con estilos elegantes. Has configurado, enlazado y desplegado de forma profesional una solución real e interactiva utilizando las bases de la virtualización ligera y los contenedores Docker.

RECURSOS ADICIONALES

1. **Documentación Oficial de Docker (Docker Docs):** El manual definitivo, guías de instalación detalladas y referencia completa de comandos directamente desde los creadores de la plataforma. Disponible en: <https://docs.docker.com>
2. **Repositorio Oficial de Docker en GitHub:** Acceso directo al código fuente abierto del motor de Docker, componentes de ecosistema y discusiones de la comunidad técnica internacional de desarrollo. Disponible en: <https://github.com/docker/docker-ce>



3. **Catálogo Público de Docker Hub:** Repositorio y biblioteca en la nube que reúne miles de imágenes oficiales listas para descargar de bases de datos, lenguajes de programación y servidores web optimizados. Disponible en: <https://hub.docker.com>